

Iage práctica 2

Héctor Aldao and Guillermo García

Facultad de Informática de Coruña

Resumen Entrenamiento de modelos de spark-mllib con datos financieros en sistemas de ficheros distribuidos virtuales.

Keywords: spark-mllib · bolsa · regresión.

1. Organización del código

El código final está organizado en dos scripts principales: uno para el problema de regresión y otro para el problema de clasificación. Ambos comparten casi toda la parte inicial del flujo de trabajo, ya que parten del mismo conjunto de datos y realizan una preparación muy similar antes de entrenar los modelos.

El primer archivo se centra en la predicción numérica del retorno futuro de una acción. En concreto, el objetivo es estimar el retorno logarítmico a 20 días, que se calcula como:

$$LogReturn20 = \log \left(\frac{Close_{t+20}}{Close_t} \right)$$

El segundo archivo reutiliza prácticamente la misma preparación de datos, pero cambia el objetivo del problema: en lugar de predecir el valor exacto del retorno, convierte el problema en una clasificación binaria. Para ello, se crea una variable que vale 1 si el retorno futuro a 20 días es positivo y 0 en caso contrario.

La estructura general de los dos programas es la siguiente:

1. Se leen los argumentos de ejecución, como el número de particiones, el nombre de la aplicación y el directorio de salida.
2. Se detecta automáticamente el *master* de Spark. Esto permite ejecutar el programa tanto en local como en el entorno distribuido de las máquinas virtuales.
3. Se crea una sesión de Spark.
4. Se cargan todos los archivos de acciones desde el directorio `data/Stocks`. Cada archivo `.txt` se interpreta como una acción distinta, y el símbolo de la acción se extrae del nombre del fichero.
5. Se limpian los datos, eliminando filas con precios inválidos, volúmenes negativos o inconsistencias básicas entre apertura, máximo, mínimo y cierre.
6. Se eliminan acciones con historiales demasiado cortos, manteniendo solo aquellas con al menos 60 observaciones.
7. Se añaden variables temporales cíclicas a partir de la fecha, como mes, día del mes y día de la semana codificados con seno y coseno.

8. Se calcula la variable objetivo: retorno logarítmico a 20 días en regresión, y clase positiva/negativa en clasificación.
9. Se generan variables de mercado adicionales: retornos pasados, volatilidades, medias móviles, ratios de volumen, rangos intradía y medidas de posición reciente.
10. Se eliminan las filas que quedan con nulos debido al uso de retardos, ventanas móviles o valores futuros.
11. Se divide el conjunto de datos temporalmente en entrenamiento, validación y test.
12. Se entrenan distintos modelos con búsqueda manual de hiperparámetros en un grid-search.
13. Se evalúan los modelos y se guardan los resultados en archivos CSV dentro del directorio de salida.

La diferencia entre ambos scripts es los modelos utilizados.

En regresión se entrenan modelos de `LinearRegression` y `GeneralizedLinearRegression`. Intentamos también un `RandomForestRegressor`, pero lo acabamos dejando comentado. También se evalúa una línea base que siempre predice retorno cero, lo que nos sirvió para ver que la regresión era bastante deficiente.

En clasificación se entrenan `LogisticRegression`, `RandomForestClassifier` y `GBTCClassifier`, junto con una línea base que siempre predice la clase mayoritaria.

2. Planteamiento

El conjunto de datos está formado por ficheros de acciones. Cada archivo tiene extensión `.txt`, aunque internamente sigue una estructura de CSV con cabecera. Cada fila representa una fecha concreta y contiene, entre otros campos, el precio de apertura, el máximo, el mínimo, el cierre, el volumen y el interés abierto. A partir del nombre del archivo se obtiene el símbolo de la acción, de forma que Spark puede leer todos los ficheros de manera conjunta y, aun así, conservar a qué acción pertenece cada fila.

En una primera fase de exploración se comprobó que los datos estaban bastante completos: no aparecían nulos relevantes en las columnas principales de precios y volumen. Sin embargo, sí se observaron algunos problemas como que había archivos vacíos o sin información útil, y no todas las acciones empezaban ni terminaban en las mismas fechas. Esto es importante porque, al trabajar con series temporales, no basta con juntar todas las filas: hay que tener en cuenta el orden temporal de cada acción y evitar usar información del futuro para predecir el pasado.

La primera transformación de los datos que teníamos clara era que queríamos que en la información temporal hubiera periodicidad. Por ello hicimos una codificación cíclica de la información del mes y el día.

El primer planteamiento fue formular el problema como una regresión. En vez de intentar predecir directamente el precio futuro de cierre, se decidió predecir el

retorno a 20 días. Esta decisión es muy típica porque los precios absolutos de las acciones no son directamente comparables entre empresas distintas: una acción puede cotizar a 5 dólares y otra a 500, sin que eso signifique que invertir en la segunda sea “mejor”. El retorno, en cambio, mide el cambio relativo y permite tratar todas las acciones de forma más homogénea.

Inicialmente se probaron transformaciones sencillas de los datos de entrada, usando retornos pasados a 3, 5 y 10 días para intentar predecir el retorno futuro a 20 días. La idea era comprobar si el comportamiento reciente de una acción podía contener información suficiente para anticipar su evolución a corto o medio plazo. Sin embargo, los resultados fueron muy malos. Incluso al probar muchas combinaciones de hiperparámetros mediante búsqueda en rejilla, los modelos apenas mejoraban una predicción trivial.

Posteriormente se amplió el conjunto de características. En los scripts finales se incorporan variables temporales, retornos pasados a diferentes ventanas, volatilidades históricas, ratios de medias móviles, información intradía, variables de volumen, drawdowns y medidas de posición del precio respecto a máximos y mínimos recientes. Además, se corrigió el planteamiento de evaluación para que el conjunto de test representase datos futuros de mercado: el entrenamiento se realiza con fechas anteriores a 2015, la validación con datos entre 2015 y 2016, y el test con datos desde 2016 en adelante.

Finalmente, al comprobar que la regresión seguía ofreciendo resultados muy débiles, se reformuló el problema como clasificación. En este caso ya no se intenta predecir cuánto va a subir o bajar una acción, sino únicamente si el retorno futuro a 20 días será positivo o no. Es un problema más sencillo, porque solo hay que acertar la dirección del movimiento, no su magnitud exacta.

3. Resultados

Los primeros resultados de regresión muestran que los modelos apenas eran capaces de explicar la variación del retorno futuro. En la Tabla 1 se incluye una selección de ejecuciones representativas. No se muestran todas las combinaciones probadas, sino una muestra de las más relevantes para ver el comportamiento general.

Modelo	regParam	elasticNet	maxIter	RMSE val.	RMSE test	MAE test	R^2 test
GLR	0.005	–	50	0.130892	0.130566	0.080477	0.003387
GLR	0.010	–	50	0.130893	0.130567	0.080467	0.003380
GLR	0.020	–	100	0.130895	0.130569	0.080452	0.003349
GLR	0.030	–	50	0.130899	0.130572	0.080441	0.003301
GLR	0.100	–	100	0.130930	0.130602	0.080413	0.002846
LR	0.010	0.5	–	0.131089	0.130757	0.080489	0.000474
LR	0.100	0.5	–	0.131120	0.130788	0.080521	-0.000000
LR	0.010	1.0	–	0.131120	0.130788	0.080521	-0.000000

Cuadro 1. Selección de resultados representativos de la ejecución de regresión. GLR corresponde a *Generalized Linear Regression* y LR a *Linear Regression*. Solo se muestra una parte de las combinaciones probadas.

La conclusión principal de estos resultados es que la regresión no consiguió aprender una señal útil. Los valores de R^2 están muy cerca de cero, e incluso en algunos casos son ligeramente negativos. Esto significa que el modelo no mejora de forma apreciable una predicción muy simple, como asumir que el retorno futuro esperado es aproximadamente cero.

Esto también se observó al comparar con una línea base que siempre predice ausencia de cambio. En este contexto, predecir retorno cero no es una idea absurda: en horizontes relativamente cortos, los retornos financieros suelen tener mucho ruido, una media cercana a cero y una varianza elevada. Por tanto, si las variables disponibles no contienen una señal clara, el modelo tiende a hacer predicciones muy conservadoras, cercanas al promedio. En la práctica, esto provoca que el RMSE y el MAE sean muy parecidos a los de una estrategia que simplemente predice que la acción se quedará igual.

Además, el hecho de que aumentar la regularización o cambiar el número de iteraciones apenas modifique las métricas indica que el problema no estaba principalmente en la elección exacta de hiperparámetros. Más bien parece que la información disponible en las características iniciales no era suficiente para predecir con precisión la magnitud del retorno futuro. Es decir, el modelo no fallaba solo por falta de ajuste, sino porque el objetivo de regresión era demasiado ruidoso y difícil para el tipo de datos y modelos utilizados.

Tras añadir más información al conjunto de entrada, el problema se planteó también como clasificación binaria. La variable objetivo pasa a valer 1 cuando el retorno a 20 días es positivo y 0 cuando no lo es. La Tabla 2 recoge una selección de resultados variados y representativos.

Modelo	Acc. val.	F1 val.	Acc. test	F1 test	AUC test	Parámetro 1	Parámetro 2
GBT	0.4923	0.4586	0.5623	0.5349	0.5535	depth=5	iter=20
GBT	0.4899	0.4565	0.5618	0.5405	0.5576	depth=5	iter=50
GBT	0.4847	0.4296	0.5725	0.5315	0.5581	depth=3	iter=20
RF	0.4865	0.4211	0.5783	0.5269	0.5624	trees=50	depth=10
RF	0.4862	0.4207	0.5788	0.5276	0.5634	trees=20	depth=10
RF	0.4755	0.3736	0.5847	0.4910	0.5597	trees=20	depth=5
LogReg	0.4887	0.4061	0.5792	0.5142	0.5431	reg=0.01	enet=0.0
LogReg	0.4782	0.3701	0.5832	0.4925	0.5442	reg=0.10	enet=0.0
LogReg	0.4657	0.3251	0.5819	0.4572	0.5387	reg=0.01	enet=1.0
Baseline	0.4551	0.2847	0.5779	0.4233	—	clase=1	—

Cuadro 2. Selección de resultados de clasificación. Los resultados mostrados son solo una porción de los experimentos realizados, escogida para representar varios modelos y configuraciones.

Los resultados de clasificación son mejores que los de regresión en el sentido de que el problema ya no parece completamente inabordable. Algunos modelos superan claramente a la línea base en F1, especialmente los modelos de Gradient Boosted Trees y Random Forest. Sin embargo, los resultados siguen sin ser especialmente destacables.

El mejor caso mostrado en validación es un *GBTClassifier*, con un F1 de validación de 0.4586 y un F1 de test de 0.5349. En test, algunos modelos alcanzan

una accuracy cercana a 0.58, pero esta cifra debe interpretarse con cuidado, porque la propia línea base de clase mayoritaria ya obtiene una accuracy de 0.5779. Es decir, una accuracy aparentemente aceptable puede deberse en parte al desequilibrio entre clases o al hecho de que en el periodo de test haya más ejemplos de una clase que de otra.

Por eso resulta más útil mirar métricas como F1 y AUC. En ese sentido, los modelos sí mejoran a la línea base, pero solo de forma moderada. Los valores de AUC se sitúan en torno a 0.54–0.56, lo que indica cierta capacidad predictiva, aunque bastante débil. Un modelo realmente útil debería estar bastante más alejado de 0.5, que representa un comportamiento prácticamente aleatorio.

En conjunto, los experimentos sugieren que predecir retornos financieros a 20 días con estos modelos sencillos y estas variables es una tarea difícil. La regresión no logró capturar bien la magnitud del movimiento futuro, probablemente porque el retorno exacto está dominado por ruido y por factores externos que no aparecen en el dataset. La clasificación, al simplificar el objetivo a subida o bajada, consiguió resultados algo mejores, pero todavía modestos.

La principal mejora metodológica respecto al planteamiento inicial fue usar una división temporal realista. Separar entrenamiento, validación y test por fechas evita que el modelo se beneficie indirectamente de información futura y hace que la evaluación sea más parecida a un escenario real de predicción de mercado. Aunque esto suele empeorar las métricas respecto a una partición aleatoria, ofrece una estimación más honesta del rendimiento del modelo.

Como conclusión final, el trabajo permitió comprobar que Spark MLlib es adecuado para procesar muchos ficheros de acciones de forma distribuida, generar variables mediante ventanas temporales y entrenar varios modelos sobre un volumen grande de datos. Sin embargo, desde el punto de vista predictivo, los resultados muestran que el problema financiero planteado es complejo y que los modelos utilizados solo encuentran una señal débil. La clasificación binaria fue más razonable que la regresión directa del retorno, pero aun así los resultados no son suficientemente buenos como para considerar que el sistema tenga una capacidad predictiva ni cerca de decente.